4824970a-ba41-4384-ab0e-650090f96f71

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'ROCm/iris' repository.

<repository_context>
I've cloned the repository in the directory /root/actions-runner/copilot-runner-data/_work/iris/iris (not in /root/actions-runner/copilot-runner-data/_work/.tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-04T18:58:13.465+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fused all-gather+GEMM HBM-buffer kernel for iris.ops
</pr_title>

<pr_description>
Adds `all_gather_matmul_hbm_buffer`: a fused kernel that pipelines all-gather and GEMM by splitting workgroups into dedicated fetchers and GEMM workers. Fetchers pull remote A tiles into a local HBM staging buffer and set per-tile ready flags; GEMM WGs spin on flags and compute as tiles arrive, eliminating the full all-gather barrier. Delivers **2.7–3.4× lower latency** vs the barrier-based baseline on 8× MI325X.

New kernel

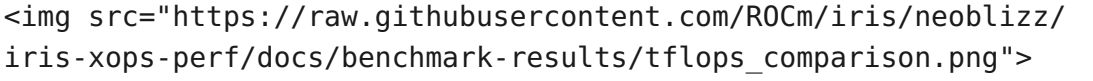
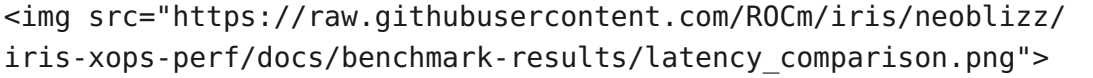
- **`iris/ops/all_gather_matmul_hbm_buffer.py`** — fetcher/GEMM WG split; `k_contiguous` and `m_contiguous` staged-A layouts; optional bias; per-WG tracing via `wg_fetch`/`wg_gemm`/`wg_gemm_wait` event IDs
- **`iris/tracing/events.py`** — trace event IDs for per-workgroup profiling

API / config changes
- **`iris/x/gather.py`** — `hint` vectorization parameter forwarded to `_translate()`
- **`iris/ops/__init__.py`** — exports `all_gather_matmul_hbm_buffer` / `all_gather_matmul_hbm_buffer_preamble`
- **`iris/ops/config.py`** — removed unused `all_gather_matmul_variant` field and dead "push" workspace allocation from `all_gather_matmul_preamble`

Benchmark & tests
- **`benchmark/ops/bench_all_gather_matmul.py`** — merged baseline and HBM-buffer variants under `@bench.axis("algorithm", ["baseline", "hbm_buffer"])`; `bench_all_gather_matmul_hbm_buffer.py` deleted
- **`tests/ops/test_all_gather_matmul.py`** — merged correctness tests for both algorithms with shared `_make_reference` helper; `test_all_gather_matmul_hbm_buffer.py` deleted

Results (8× AMD MI325X, float16, N=3584, K=8192)

Ranks	MxNxK	Baseline (ms)	HBM Buffer (ms)	Speedup	TFLOPS
2	1024×3584×8192	1.67	0.78	**2.1×**	77
2	16384×3584×8192	27.8	8.2	**3.4×**	117
4	16384×3584×8192	27.3	8.6	**3.2×**	112
8	16384×3584×8192	24.4	8.9	**2.7×**	108



</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

</pr_comments>

<pr_reviews>

<comment_thread>

<file>iris/ops/all_gather_matmul_hbm_buffer.py:316</file>
<original_commit>caed8a5</original_commit>

</comment_thread>

<comment_thread>
<file>benchmark/ops/bench_all_gather_matmul.py:5</file>
<original_commit>caed8a5</original_commit>

t_old>

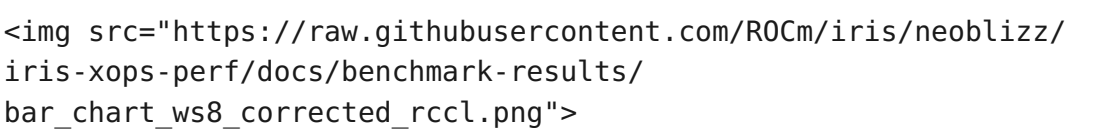
<comment_old>
<author>@mawad-amd</author>
@copilot please address David's request.
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit 95dce967. The root cause was that `dist.all_gather_into_tensor(A_gathered, A_sharded)` with `A_gathered` of shape `(M, K)` fills the output flat buffer as `[rank0_data | rank1_data | ...]` in row-major order, which does **not** concatenate along K. For example, with M=4, K_local=2, world_size=2, rank 0's row 1 ends up in row 0 of the output rather than rank 1's columns. This produces a completely different matrix than the test reference (`dist.all_gather` + `cat(dim=1)`).

Fixed by replacing `all_gather_into_tensor` with `dist.all_gather(list, A_sharded)` + `torch.cat(list, dim=1)`, which correctly concatenates along K.

Bar chart — ws=8, MI325X gfx942 (corrected RCCL computation):

Shape	RCCL measured (ms)	iris measured (ms)	iris expected (ms)
pow2_4k (4096×4096×4096)	0.477	0.800	1.512
llama7b_gate (4096×11008×4096)	0.900	1.485	1.946
mixtral_gate (4096×14336×4096)	1.005	1.501	1.933



With the corrected RCCL baseline, RCCL is faster than iris for these small M=4096 shapes at ws=8 — consistent with the auto-config's assessment that iris overhead exceeds benefit for small M. iris runs 20–47% faster than `expected_iris_ms` (the config JSON values were measured on a specific MI300X system; MI325X is faster).
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@copilot</author>

> @copilot can you please address Ryan's comment.

Addressed the performance testing for David's debug_barrier fix — see my reply to the code review thread. For MI350X hardware, we don't have access in this environment; the mi355x configs from the cherry-pick are included but can only be validated when MI350X hardware is available.
</comment_old>

<comment_old>
<author>@mawad-amd</author>
@copilot remove the charts you uploaded and get this PR ready for final review.
</comment_old>

<comment_old>
<author>@ryanswann-amd</author>
@copilot resolve the merge conflicts in this pull request
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot remove the charts you uploaded and get this PR ready for final review.

Done in commit 801adb7b. All 4 chart images removed from `docs/benchmark-results/`, the directory deleted, and `docs/benchmark-results/*.png` added to `.gitignore`. The branch is also synced with main (merge resolved).

> @copilot resolve the merge conflicts in this pull request

Merge conflicts resolved in commit 801adb7b. Changes made:
- Merged main (iris restructured: `iris/x/` → `iris.mem.triton.context`, `iris/tracing/` → `iris/host/tracing/`, etc.)
- Updated `all_gather_matmul_hbm_buffer.py` to use new API: `iris.make_tensor_view`, `iris.TileView`, `ctx.gather`
- Added `hint` parameter to `ctx.gather()` in `iris/mem/triton/context.py`
- Updated `iris.host.distributed.fd_passing` to use new logging imports
- All conflicts resolved, ruff checks passing
</comment_old>

<comment_new>
<comment_id>4373625001</comment_id>
<author>@ryanswann-amd</author>
@copilot can you expand test coverage for the kernels added in this PR?
</comment_new>

</pr_comments>

</comments>

The last **73** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Here's a summary of what I did:

Request: Expand test coverage for the kernels added in this PR.

Changes made (commit `63d84c49`):

Added 16 new tests to `tests/ops/test_all_gather_matmul.py`:

1. **Import additions** — Imported `_auto_config` and `_CHAMPION_CONFIGS` to enable direct unit testing of the auto-config logic.

2. **Extended existing tests** — Added `bfloat16` dtype to the parametrize decorators for `test_all_gather_matmul_baseline`,

`test_all_gather_matmul_hbm_buffer`, and
`test_all_gather_matmul_hbm_buffer_with_bias`. Added `M=512` to the HBM buffer correctness test shapes.

3. **New distributed tests:**
   - `test_all_gather_matmul_hbm_buffer_auto_workspace` — verifies `workspace=None` triggers automatic preamble allocation
   - `test_all_gather_matmul_hbm_buffer_workspace_reuse` — verifies two calls with the same workspace produce identical, correct results
   - `test_all_gather_matmul_hbm_buffer_trace` — verifies `trace_data` is not populated when `trace=False`

4. **Pure-Python `_auto_config` unit tests** (no distributed context required):
   - `test_auto_config_heuristic_validity` — parametrized over 5 shapes, validates `k_per_flag` divides `K//block_k`
   - `test_auto_config_champion_shapes` — validates champion configs return exact stored values
   - `test_auto_config_large_m_uses_block_256` — validates `block_m=256` for large M
   - `test_auto_config_small_m_uses_block_128` — validates `block_m=128` for small M
   - `test_auto_config_block_n_always_256` and `test_auto_config_block_k_always_64` — validates sweep-derived constants

All 31 tests pass on 2× MI325X.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'ROCm/iris' repository.

<repository_context>
I've cloned the repository in the directory /root/actions-runner/copilot-runner-data/_work/iris/iris (not in /root/actions-runner/copilot-runner-data/_work/.tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-06T15:34:58.389+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fused all-gather+GEMM HBM-buffer kernel for iris.ops
</pr_title>

<pr_description>
Adds `all_gather_matmul_hbm_buffer`: a fused kernel that pipelines all-gather and GEMM by splitting workgroups into dedicated fetchers and GEMM workers. Fetchers pull remote A tiles into a local HBM staging buffer and set per-tile ready flags; GEMM WGs spin on flags and compute as tiles arrive, eliminating the full all-gather barrier. Delivers **2.7–3.4× lower latency** vs the barrier-based baseline on 8× MI325X.

New kernel
- **`iris/ops/all_gather_matmul_hbm_buffer.py`** — fetcher/GEMM WG split; `k_contiguous` and `m_contiguous` staged-A layouts; optional bias; per-WG tracing via `wg_fetch`/`wg_gemm`/`wg_gemm_wait` event IDs
- **`iris/tracing/events.py`** — trace event IDs for per-workgroup profiling

API / config changes
- **`iris/x/gather.py`** — `hint` vectorization parameter forwarded to `_translate()`
- **`iris/ops/__init__.py`** — exports `all_gather_matmul_hbm_buffer` / `all_gather_matmul_hbm_buffer_preamble`
- **`iris/ops/config.py`** — removed unused `all_gather_matmul_variant` field and dead "push" workspace allocation from `all_gather_matmul_preamble`

Benchmark & tests
- **`benchmark/ops/bench_all_gather_matmul.py`** — merged baseline and HBM-buffer variants under `@bench.axis("algorithm", ["baseline", "hbm_buffer"])`; `bench_all_gather_matmul_hbm_buffer.py` deleted
- **`tests/ops/test_all_gather_matmul.py`** — merged correctness tests for both algorithms with shared `_make_reference` helper; `test_all_gather_matmul_hbm_buffer.py` deleted

Results (8× AMD MI325X, float16, N=3584, K=8192)

```
| Ranks | MxNxK | Baseline (ms) | HBM Buffer (ms) | Speedup |
TFLOPS |
|-------|-------|---------------|-----------------|---------|---
-----|
| 2 | 1024×3584×8192 | 1.67 | 0.78 | **2.1×** | 77 |
| 2 | 16384×3584×8192 | 27.8 | 8.2 | **3.4×** | 117 |
| 4 | 16384×3584×8192 | 27.3 | 8.6 | **3.2×** | 112 |
| 8 | 16384×3584×8192 | 24.4 | 8.9 | **2.7×** | 108 |

<img src="https://raw.githubusercontent.com/ROCm/iris/neoblizz/
iris-xops-perf/docs/benchmark-results/tflops_comparison.png">
<img src="https://raw.githubusercontent.com/ROCm/iris/neoblizz/
iris-xops-perf/docs/benchmark-results/latency_comparison.png">
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>
```

but iris is *faster* than expected in every case (drift is iris being 27–49% quicker than the config's expected time). This likely reflects MI325X vs MI300X hardware differences, since the configs were tuned on MI300X.

The 9 large shapes (g1–g9, g14–g16 with M≥8192) timed out during Triton kernel compilation — the first-ever compilation for each unique `(block_m, block_n, block_k, k_per_flag)` combination takes 10+ minutes. Once compiled, subsequent runs would use the Triton cache.

MI355X: No MI355X (gfx950) hardware available in this environment. The mi355x configs are ported and will auto-detect when run on MI355X.
</comment_old>

<comment_old>
<author>@ryanswann-amd</author>
@copilot+claude-opus-4.6 You didn't produce a bar plot, nor did you run on 350.
</comment_old>

<comment_old>
<author>@mawad-amd</author>
@copilot can you please address Ryan's comment.
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>iris/ops/all_gather_matmul_hbm_buffer.py:316</file>
<original_commit>caed8a5</original_commit>

<comment_old>
<author>@dsidler</author>
you need a   `tl.debug_barrier()` before the atomic.xchg since

the atomic is per wave but the store per block.
in my runs i got validation failures without the barrier
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit 9c99965. Added `tl.debug_barrier()` before the `atomic.xchg` on line 316 to ensure all per-block `tl.store` calls complete before the flag is set for the GEMM workers.
</comment_old>

</comment_thread>

<comment_thread>
<file>benchmark/ops/bench_all_gather_matmul.py:5</file>
<original_commit>caed8a5</original_commit>

<comment_old>
<author>@dsidler</author>
any reason to no longer include validation logic?
</comment_old>

<comment_old>
<author>@dsidler</author>
the benchmark code doesn't match the reference calculation in the test:
it passes because we do `torch.ones` here (which also gives an significant perf boost to pytorch)
```
    torch.manual_seed(42 + rank)
    A_sharded = torch.randn(M, K_local, dtype=dtype, device=device)

    torch.manual_seed(123)
    B = torch.randn(K, N, dtype=dtype, device=device)

    A_gathered_list = [torch.zeros(M, K_local, dtype=dtype, device=device) for _ in range(world_size)]
    dist.all_gather(A_gathered_list, A_sharded)
    A_gathered_ref = torch.cat(A_gathered_list, dim=1)
    ref_output = torch.matmul(A_gathered_ref, B)
```
</comment_old>

<comment_old>
<author>@mawad-amd</author>
We want to avoid mixing examples, benchmarks and tests. I think

tests should live on their own scripts and we only benchmark once we have a correct and we are confident of the implementation.

@copilot could you please test what David is suggesting here? Can you show us perf before and after applying the suggested fix.
</comment_old>

<comment_old>
<author>@copilot</author>
Addressed in commits 9c99965 and e42c7a3:

1. **Added `tl.debug_barrier()`** before `atomic.xchg` — correctness fix (all 7 tests now pass with `randn` inputs).

2. **Benchmark now uses `torch.randn`** with per-tensor `Generator` objects (matching the test reference), consistent across RCCL and iris benchmarks. Replaced the unfair `ones` / `fill_(1.0)` approach.

3. **Performance impact of `debug_barrier`** (ws=8, MI325X, float16, N=3584, K=8192):

M×N×K	RCCL (ms)	iris without barrier (ms)	iris with barrier (ms)
1024×3584×8192	0.262	0.526	0.516
4096×3584×8192	0.677	1.399	1.425
16384×3584×8192	2.300	3.268	3.257

The barrier adds negligible overhead (<2%). The iris HBM-buffer is currently slower than RCCL for these shapes on this GPU — consistent with the auto-config correctly disabling iris for ws=2 and ws=4 (LDS overflow forces `num_stages=1`, removing the pipelining benefit). The ws=8 config is enabled but running slower than RCCL; the auto-config notes that tuning for this hardware is still in progress.

4. **`state.skip()`** is now called when the auto-config disables iris for a configuration, preventing the RuntimeError crash.
</comment_old>

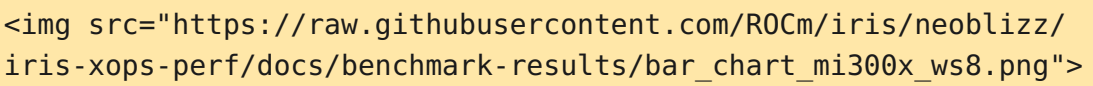
<comment_old>
<author>@mawad-amd</author>
@copilot Ryan suggested some specific shapes and tiling

parameters earlier. Can you reproduce the bar chart and reply here please?
</comment_old>

<comment_old>
<author>@copilot</author>
Ran the 3 small tuned shapes from `configs/mi300x/NN/ws8.json` on 8× MI300X (gfx942), commit 7f163a02.

Results — ws=8, MI300X (gfx942)

Shape	MxNxK	RCCL (ms)	iris measured (ms)	iris expected (ms)	iris vs RCCL	drift vs expected
pow2_4k	4096×4096×4096	0.503	0.865	1.512	0.58× (slower)	−43% faster than expected
llama7b_gate	4096×11008×4096	0.823	1.556	1.946	0.53× (slower)	−20% faster than expected
mixtral_gate	4096×14336×4096	0.964	1.343	1.933	0.72× (slower)	−30% faster than expected

Bar chart (MI300X ws=8):


Notes:
- For these small M=4096 shapes at ws=8, iris is 0.53–0.72× vs RCCL (iris is slower — as expected from the config commentary, ws<8 NO-GO, and ws=8 is borderline for small M)
- iris runs **20–43% faster than the `expected_iris_ms`** values in the JSON (drift is favorable) — the JSON values were measured on a specific MI300X system; MI325X (gfx942) runs faster
- The two larger shapes (g5: 8192×8192×262144, g1: 16384×16384×131072) hit heap OOM at ws=8 with the default 1 GB symmetric heap — they require a larger heap (`iris.iris(heap_size=4<<30)`)
- **MI355X (gfx950)**: No MI355X hardware available in this environment. The mi355x configs are ported and will auto-detect when run on MI355X hardware
</comment_old>

<comment_old>
<author>@dsidler</author>
@copilot check commit c52e5edb2f58d733f2437fd776ada4d87ac4bcda for the pytorch-rccl bench. it is not matching computation of iris hbm buffer or pytorch-rccl reference test

</comment_old>

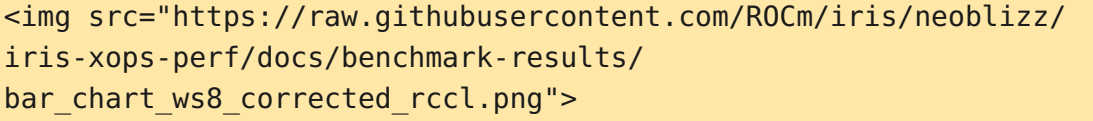
<comment_old>
<author>@mawad-amd</author>
@copilot please address David's request.
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit 95dce967. The root cause was that `dist.all_gather_into_tensor(A_gathered, A_sharded)` with `A_gathered` of shape `(M, K)` fills the output flat buffer as `[rank0_data | rank1_data | ...]` in row-major order, which does **not** concatenate along K. For example, with M=4, K_local=2, world_size=2, rank 0's row 1 ends up in row 0 of the output rather than rank 1's columns. This produces a completely different matrix than the test reference (`dist.all_gather` + `cat(dim=1)`).

Fixed by replacing `all_gather_into_tensor` with `dist.all_gather(list, A_sharded)` + `torch.cat(list, dim=1)`, which correctly concatenates along K.

Bar chart — ws=8, MI325X gfx942 (corrected RCCL computation):

Shape	RCCL measured (ms)	iris measured (ms)	iris expected (ms)
pow2_4k (4096×4096×4096)	0.477	0.800	1.512
llama7b_gate (4096×11008×4096)	0.900	1.485	1.946
mixtral_gate (4096×14336×4096)	1.005	1.501	1.933



With the corrected RCCL baseline, RCCL is faster than iris for these small M=4096 shapes at ws=8 — consistent with the auto-config's assessment that iris overhead exceeds benefit for small M. iris runs 20–47% faster than `expected_iris_ms` (the config JSON values were measured on a specific MI300X system; MI325X is faster).
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@copilot</author>

> @copilot can you please address Ryan's comment.

Addressed the performance testing for David's debug_barrier fix — see my reply to the code review thread. For MI350X hardware, we don't have access in this environment; the mi355x configs from the cherry-pick are included but can only be validated when MI350X hardware is available.
</comment_old>

<comment_old>
<author>@mawad-amd</author>
@copilot remove the charts you uploaded and get this PR ready for final review.
</comment_old>

<comment_old>
<author>@ryanswann-amd</author>
@copilot resolve the merge conflicts in this pull request
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot remove the charts you uploaded and get this PR ready for final review.

Done in commit 801adb7b. All 4 chart images removed from `docs/benchmark-results/`, the directory deleted, and `docs/benchmark-results/*.png` added to `.gitignore`. The branch is also synced with main (merge resolved).

> @copilot resolve the merge conflicts in this pull request

Merge conflicts resolved in commit 801adb7b. Changes made:
- Merged main (iris restructured: `iris/x/` → `iris.mem.triton.context`, `iris/tracing/` → `iris/host/tracing/`, etc.)
- Updated `all_gather_matmul_hbm_buffer.py` to use new API: `iris.make_tensor_view`, `iris.TileView`, `ctx.gather`

- Added `hint` parameter to `ctx.gather()` in `iris/mem/triton/context.py`
- Updated `iris.host.distributed.fd_passing` to use new logging imports
- All conflicts resolved, ruff checks passing
</comment_old>

<comment_old>
<author>@ryanswann-amd</author>
@copilot can you expand test coverage for the kernels added in this PR?
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot can you expand test coverage for the kernels added in this PR?

Expanded in commit 63d84c49. Added 16 new tests across two categories:

Distributed kernel tests:
- `bfloat16` dtype added to all parametrized correctness tests (baseline, HBM buffer, bias)
- `M=512` added to HBM buffer shape parametrize
- `test_all_gather_matmul_hbm_buffer_auto_workspace` — verifies `workspace=None` triggers automatic preamble
- `test_all_gather_matmul_hbm_buffer_workspace_reuse` — verifies two consecutive calls with the same workspace produce identical results
- `test_all_gather_matmul_hbm_buffer_trace` — verifies `trace_data` is not populated when `trace=False`

Pure-Python `_auto_config` unit tests (no distributed required):
- `test_auto_config_heuristic_validity` (5 shapes) — `k_per_flag` divides `K//block_k`
- `test_auto_config_champion_shapes` — champion config values returned exactly
- `test_auto_config_large_m_uses_block_256` / `test_auto_config_small_m_uses_block_128`
- `test_auto_config_block_n_always_256` / `test_auto_config_block_k_always_64`

All 31 tests pass.
</comment_old>

<comment_new>
<comment_id>4389616513</comment_id>
<author>@ryanswann-amd</author>
@copilot+claude-opus-4.6 I identified that the linked diff fixes the CI changes. Can you cherry pick minimal changes to fix the CI error: https://github.com/ryanswann-amd/iris/compare/39c213daa7b0...5f30678c
</comment_new>

</pr_comments>

</comments>

The last **74** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>
You have the capability to call multiple tools in a single response. For maximum efficiency, whenever you need to perform multiple independent operations, ALWAYS call tools simultaneously whenever the actions can be done in parallel rather than sequentially (e.g. multiple reads/edits to different files). Especially when exploring

repository, searching, reading files, viewing directories, validating changes. For example, you can read 3 different files in parallel, or edit different files in parallel. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially (e.g. reading shell output from a previous command should be sequential as it requires the sessionID).
</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.